Course : Individual Software Development Process' 2026

Iteration Report : Iteration 4

By Ultrasmooth

Chosen Irl Challenge : Project A - Lab Data Management



Jirat Kiattrairong	6810545506
Napakhet Namsrioun	6810545727
Panyasiri Aimngern	6810545743
Panisara Niyathirakul	6810545751

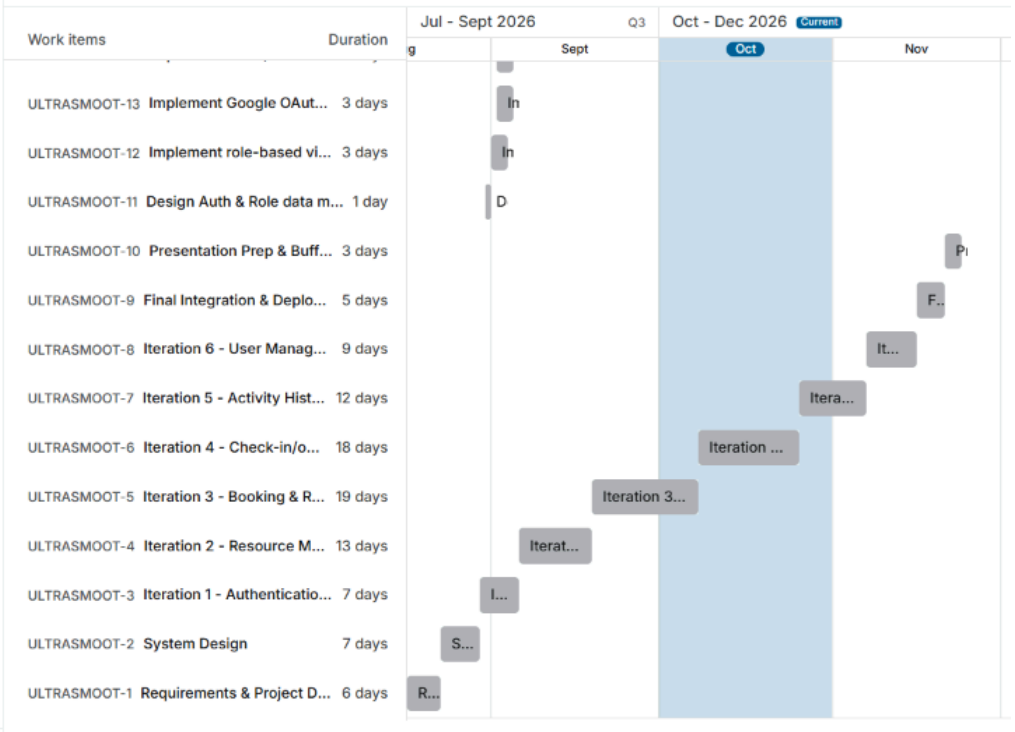
Table of Contents

1. Project Snapshot..... 1
1.1 Project-Wide Gantt Chart.....1
1.2 Current Iteration Gantt Chart.....2
2. Sprint/Board Status Snapshot.....3
3. Scope Addition/Change Log..... 6
4. Retrospective..... 6
5. Testing & Quality Notes.....7
6. Project Risk Register.....9
7. Individual Contribution Log..... 10

1. Project Snapshot

Project Repository : https://github.com/Ultrasmoot/Ultrasmooth

1.1 Project-Wide Gantt Chart



Critical Path :

Requirements & Project Documentation

↓

System Design

↓

Iteration 1 - Authentication & Access (SRS-11, SRS-12)

↓

Iteration 2 - Resource Management (SRS-1, SRS-2)

↓

Iteration 3 - Booking & Request Management (SRS-3, SRS-4, SRS-5)

↓

Iteration 4 - Check-in/Check-out & Maintenance (SRS-10, SRS-6, SRS-7)

↓

Iteration 5 - Activity Log & Dashboard (SRS-8, SRS-9)

↓

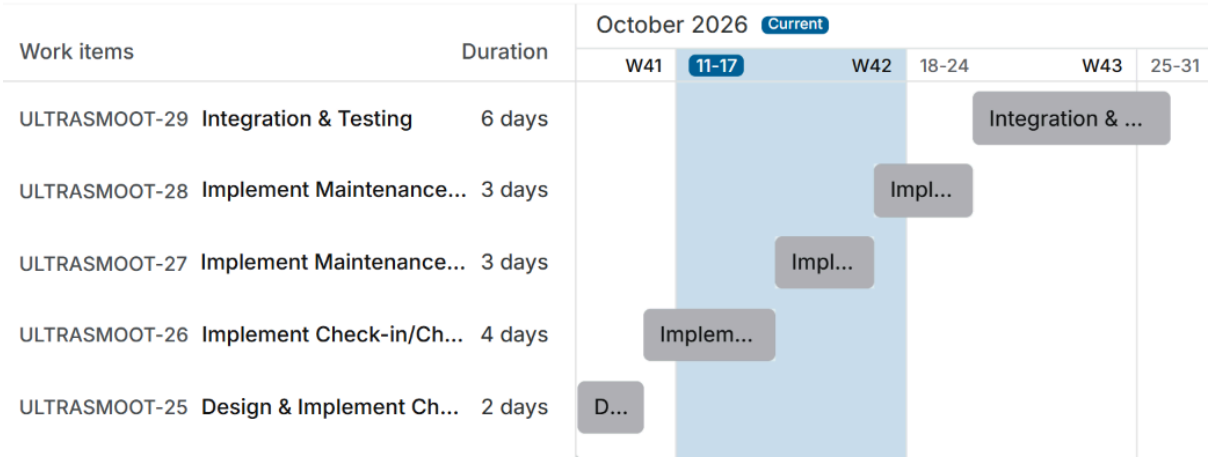
Iteration 6 - User Management (SRS-13)

↓

Final Integration & Deployment

Critical Path Duration : 99 calendar days

1.2 Current Iteration Gantt Chart



Critical Path :

Design & Implement Check-in/Check-out Structure
↓
Implement Check-in/Check-out Functions
↓
Implement Maintenance Issue Reporting
↓
Implement Maintenance Resolution & Resource Restrictions
↓
Integration & Testing

Critical Path Duration : 18 calendar days

2. Sprint/Board Status Snapshot

Iteration Goal : The goal of Iteration 4 was to connect the booking flow from Iteration 3 to what really happens in the lab. Any user can report a problem with a resource, and an administrator can confirm, edit and resolve it, so the status of a resource always matches its real condition. Students can check in to their own approved booking with a 6 digit one-time code sent to their registered e-mail, and check out when they are done. This supports Sub-Goals SG-3 (Issues & Maintenance) and SG-5 (Resource Availability & Controlled Usage) from our SRS, through SRS-6, SRS-7 and SRS-10.

During this iteration, the team focused on three user stories :

- US-6 : Report a resource issue
- US-7 : Resolve maintenance records
- US-10 : Check in/out a resource

The implemented features include a maintenance report with validation (required description of up to 500 characters, no future date, no archived resource), a review step where an administrator confirms a report or rejects it with a reason and editing and resolving of records. A resource returns to Available only when its last active issue is resolved.

For check-in, the student requests a 6-digit code that is sent by e-mail. The code expires after 5 minutes, allows 3 attempts and can be used once. The check-in window opens 15 minutes before the booking starts, and a resource under Maintenance cannot be checked in to.

Capacity Declared : 3 user stories (US-6, US-7 and US-10), the same as Iteration 3. We kept the number at three because Iteration 3 delivered three stories with nothing carried over, and because the three stories share the same resource status (Available, In Use and Maintenance) so the status rules could be defined once.

User Story	Description	Owner	Status	Within Capacity (Y/N)	Notes
US-6	As a Lab Administrator, I want to report a resource issue so broken resources are documented and blocked from booking.	Jirat Kiattrairong (backend and maintenance page) Panisara Niyathirakul (Home list and resource-card shortcuts)	Done	Y	Every role can report. A report from a non-admin is saved as "Reported" and does not change the resource until an administrator confirms it. An administrator's own report is confirmed straight away and the resource moves to Maintenance. The description is required (up to 500 characters), the date cannot be in the future, and archived resources cannot be reported. An administrator can confirm a report, or reject it with a required reason that the reporter can read (SRS-6).
US-7	As a Lab Administrator, I want to resolve maintenance records so resource status stays accurate.	Jirat Kiattrairong	Done	Y	Administrators can edit an “Open” or “In Progress” record (description, date and status, but not the resource). They can dismiss a report with a required reason of up to 255 characters, or resolve an issue, and the system saves who closed it and when. The resource returns to “Available” only when no other “Open” or “In Progress” issue remains for it, so one resolved issue does not free a resource that is still broken (SRS-7). The Maintenance page has three filters: “Pending review”, “In maintenance” and “History”. Administrators see all records, and other users see only the issues they reported. Confirm, dismiss, edit and resolve lock the resource row first and then the record, so

					two administrators cannot overwrite each other.
US-10	As a Ph.D. Student or Undergraduate Student, I want to check in/out a resource so its status updates automatically.	Panisara Niyathirakul (check-in logic and page) Napakhet Namsrioun (code-email and real-database tests) Panyasiri Aimngern (booking rules for an In Use resource)	Done	Y	Only Ph.D. and Undergraduate students can check in, and only to their own approved bookings, from 15 minutes before the start until the booking ends. Check-in is refused if the booking is not approved, was already checked in, or the resource is “In Use” or in “Maintenance”. The student asks for a code, and the system sends a six-digit code to the registered email address. The code is valid for five minutes and allows three wrong attempts. A new code can be requested after 30 seconds, and only the newest code works, once. Only a hash of the code is stored. A valid check-in saves the time and sets the resource to “In Use”. Check-out saves the time and sets the resource to “Available” only if it is still “In Use”, so a resource in Maintenance stays blocked (SRS-10). The “Check in/out” page lists the student’s resources in use, the bookings that can be checked in, and the recently completed bookings.

3. Scope Addition/Change Log

Item added/changed	Reason	What as dropped to compensate (one-in-one-out)
Who can report an issue (US-6, SRS-6). The proposal says Lab Managers/Administrators log resource issues. In the implementation, any signed-in user can report one. A report from a non-administrator stays in "Reported" status until an administrator confirms or dismisses it.	The person who first notices a broken resource is usually a student, not an administrator. The review step stops a wrong report from blocking a resource that works.	Nothing was dropped to make room. The administrator-only rule was replaced by the review step, and no user story was reduced.
How a student checks in (US-10, SRS-10). The proposal says the student scans or enters a resource code. In the implementation, the student enters a six-digit code that is sent to the email address of the person who made the booking.	The code shows that the person checking in is the owner of the booking, and it does not need a label on every resource.	Nothing was dropped to make room. The email code replaced scanning or entering a resource code, and no user story was reduced.
Reason for rejecting a booking. An administrator can write an optional note when rejecting a request, and the requester can see it.	Students could see "Rejected" but did not know why.	**None.** This is a small extension to Iteration 3 and is outside the three declared stories.

4. Retrospective

What went well	What didn't go well	Action items for next sprint
In Iteration 3 we promised automated tests before Iteration 4. We delivered: 188 automated tests now cover check-in, maintenance, booking rules while a resource is In Use, approval re-checks and archiving.	Real concurrency is still not tested. Our fast tests use an in-memory fake database, which cannot prove that the row locks work. Two parallel requests for the same slot, or two admins confirming the same report, are still untested.	Add concurrency tests to the real-MySQL suite (two threads, same slot, same report) before Iteration 5 starts.
We used one locking order everywhere (resource row first, then the booking or record). Booking, maintenance and check-in share it, so they cannot deadlock each other.	One defect reached review : after an UPDATE, `lastrowid` was reset to 0, so an administrator's report was saved but the request failed with a 500. It slipped through	Every new INSERT-then-UPDATE flow needs a test that reads back the saved record. Run the real-MySQL suite before each demo, not only before the final

	because the happy path was only covered by mocks.	release.
We re-check the rules at the moment of use, not only when the screen loads. For example, verifying a code checks the resource status again, because it may have gone into Maintenance while the e-mail was on its way.	The three stories took up the whole iteration, and the extra hardening work (see Scope Log) used time that we had meant to keep as a buffer.	Keep a fixed buffer of about 15% of capacity for review fixes and do not count it as story work.
The demo seed gives a student an approved booking, one open issue and one reported issue, so every flow can be shown right after `docker compose up`.	A frontend test (`session_401.test.js`) currently fails, because it looks for a comment marker that was removed from `index.html`. Three open items from Iteration 3 (professor view, shared rate limiter, concurrency tests) were not closed.	Fix the frontend test locator. Ask the stakeholder about the professor's view. Replace the in-memory rate limiter before deployment.

5. Testing & Quality Notes

Note : The team reviewed the implementation of US-6, US-7 and US-10. The Maintenance Model and the Check-in Model were reviewed for input validation, status changes, locking and transaction handling.

Automated tests. Run from `backend/` with `python -m unittest discover -s tests`: **195 tests, 185 passing and 10 skipped by default.** The skipped ones are real-MySQL integration tests that run only when a test database is available. They run in a throw-away MySQL container with `docker compose -f docker-compose.test.yml up --build --abort-on-container-exit`.

New or extended test files for this iteration:

- `test_checkin.py` (28 tests): access control (students only, someone else's booking looks like a missing one), the list split into in use / ready / done, code request and verification, and check-out.
- `test_maintenance.py` (46 tests): permissions, report validation, the Reported → Open → Resolved/Dismissed flow, editing, resource-status guards and the return-to-Available action.
- `test_booking_in_use.py` (14), `test_booking_reason.py` (13), `test_archive_resource.py` (14): the rules that connect booking, maintenance and check-in.

US-6. We tested missing or blank descriptions, a description over 500 characters, missing, invalid and future dates, unknown or archived resources, and that a non-admin report does not touch the resource. A "Reported" issue does not block booking. Non-admins see only their own reports.

US-7. We tested that confirming or resolving twice is refused, that unconfirmed reports cannot be resolved, and that the resource stays in Maintenance until its last active issue is resolved. We also tested that the resource of a record cannot be changed, that closed records are locked, and that an edit form which resends an old status cannot undo a new maintenance.

US-10. We tested each blocked case separately (not approved, already checked in, Maintenance, In Use, too early, booking ended) and that nothing is sent in those cases. Further tests: check-in opens exactly 15 minutes before the start, only the newest code works, three wrong guesses lock the code even for the right answer, expired and malformed codes are refused, and the failed-attempt counter survives the failed request. If the e-mail fails, the response is 502, no code is kept and the 30-second cooldown does not start. Check-out cannot lift a Maintenance block, and a late check-out still works.

Defects found and fixed.

1. An administrator's own maintenance report was saved but returned a 500 error, because lastrowid is reset by the following UPDATE. We now read the id straight after the INSERT. Regression test: LastRowIdRegressionTests.
2. A booking that overlapped an approved one could be approved after waiting in the queue. Approval now re-checks overlap, start time, resource status and archive state. Tests: test_booking_reason.py.
3. A resource set to In Use by hand could be double-reserved. Fixed by the In Use rule above (test_booking_in_use.py).

Review notes. The team reviewed the code for the three stories and the review items listed in the Scope Log. We checked the transaction handling, the lock order and the server-side role checks on every new endpoint.

Manual testing. We walked through the demo seed: student check-in with the logged code (e-mail is printed to the server log when SMTP is not configured), checkout, an administrator confirming and rejecting the reported laser cutter issue, and returning RES-0004 to Available.

6. Project Risk Register

Risk ID	Description	Likelihood (L/M/H)	Impact (L/M/H)	Mitigation/ Contingency	Status
R-01	Double booking or double confirmation. Row locks should serialise requests, but our fast tests use a fake database and do not prove it.	L	H	Booking, approval, maintenance and check-in lock the resource row first. Add real-MySQL concurrency tests (two threads, same slot or report) in Iteration 5.	Open
R-02	Professors have no read-only view of all booking requests, and the access they need is not confirmed by the stakeholder.	M	M	Ask the stakeholder in the next meeting, then update permissions and views.	Open
R-03	Resource availability checks were untested for In Use, Maintenance and archived resources.	M	M	Now covered by test_booking_in_use.py, test_maintenance.py and test_archive_resource.py.	Closed
R-04	The rate limiter keeps counters in memory, so they reset on restart and are not shared between workers. It now also protects the check-in endpoints.	M	M	Use Redis or Flask-Limiter before deployment. The most important check-in protection, the 3-attempt limit per code, is stored in the database, so a restart does not reset it.	Open
R-05	Date/time values are naive server-local time. A wrong server timezone would break the check-in window and past-time checks.	L	M	Keep TZ=Asia/Bangkok in Docker and --default-time-zone=+07:00 in the test database. Tests patch the clock.	Monitoring

R-06	Check-in depends on SMTP. If the mail server is down, students cannot check in.	M	H	Failed sends return a clear error and do not start the cooldown. Consider an administrator override or manual check-in in a later iteration.	Open
R-07	A student forgets to check out, so the resource stays In Use.	M	M	Later slots can still be booked after the overstayed session ends, and an administrator can reset the status. Show overstays in the Iteration 5 dashboard.	Open

7. Individual Contribution Log

Student Name	Tasks owned this sprint	Course concept(s) applied	Evidence	Self-rated effort (1-5)
Napakhet Namsrioun	**US-10 (code e-mail) :** wrote the check-in code e-mail and the shared mailer, with a timeout on every SMTP step and a certificate check on the connection. **US-10 (real-database tests) :** wrote the integration tests that run on a real MySQL, the throw-away test database setup (docker-compose.test.yml), and the tests for archiving resources. **Extra - E-mail verification (SRS-11) :** wrote migration 003, the email_verifications table, the single-use verification link, the verify and resend endpoints, and their tests. **Extra - Account security :** wrote the Google account-linking rules, the	**Transaction design :** the check-in code is saved and committed first, and the e-mail is sent afterwards, so a slow mail server never holds a lock on the resource row. **Single-use tokens :** a verification or reset link is consumed by one conditional UPDATE, so two simultaneous requests cannot both succeed. Only the SHA-256 hash of the token is stored. **Fail-safe error handling :** if the e-mail cannot be sent, the code is deleted and the 30-second cooldown does not start. Reset and resend answers are the same whether or not the account exists, so addresses cannot be discovered. **Test environment isolation :** integration tests	**ef4ea3d** services/mailer.py (check-in code e-mail) **99e9405 1ab0452** email_verifications table and migration 003 **f6b5267 1f7a508** Google sign-up, email verification, password reset and auth flow fixes **6e20db1** test_auth_followup .py **941a486** added tests in test_security.py	5

	clock-skew handling for Google sign-in, the hashed password-reset tokens, and test_auth_followup.py.	run in a throw-away database with the same timezone as the app, and they refuse to run unless the database name ends with _test.		
Panisara Niyathirakul	**US-10 (logic) :** wrote the check-in and check-out logic: the 6-digit code request, the verification (3 attempts, 5-minute expiry, single use), the 30-second resend cooldown, and the check-in window that opens 15 minutes before the start time. **US-10 (page) :** wrote the Check in/out page with three lists (in use, approved bookings, recently completed), the code dialog, the resend timer, and the reason shown next to a disabled button. **US-10 (data and tests) :** designed the checkin_codes table, added the demo booking to the seed file, and wrote test_checkin.py. **US-6/US-7 (shortcuts) :** wrote the "My Reported Issues" list on the Home page, the "Report issue" button on resource cards, and the "Return to Available" and "Unarchive" buttons for administrators. **Extra - Class year :** wrote migration 002 and the class-year field on the create-account and complete-profile forms, and showed the account name next to the avatar.	**Security by design :** only an HMAC of the code, tied to the booking and the user, is stored. The short expiry, the 3-attempt limit and single use are the real protection, because a 6-digit code alone is easy to guess. **Business rules in the Model layer (MVC) :** every reason why a check-in is blocked is decided in block_reason() in the model. The controller only turns the result into JSON. **Validation at the point of use :** verifying a code checks the booking, the resource status and the time window again, because the resource may have gone into Maintenance while the e-mail was on its way. **Concurrency control :** the resource row and then the booking row are locked with FOR UPDATE. A wrong guess is committed even though the request fails, so the attempt counter cannot be reset by retrying. **User feedback :** the server's reason, for example "Check-in opens at 09:45", is shown on the page, so the student knows what to do next.	**3ce277a fa7874a** checkin_controller.py **454a8dc** checkin_model.py (HMAC, block_reason, FOR UPDATE) **e3c613c** Rtest_checkin.py **4f2aa29 8eae246** checkin_codes table and timestamps in the schema **4d42d5a 7025374** check-in/check-out UI **f3cd8d9** migration 002 (class year) **e73d06f** “Return to Available” / maintenance status UI	5

Jirat Kiattrairong	**US-6 :** wrote the maintenance report flow. Every role can report. A report from a non-admin is saved as "Reported", and an administrator's own report is confirmed immediately. An administrator can confirm a report or reject it with a required reason. **US-7 :** wrote the edit and resolve functions, and the "return to Available" action that closes all active issues of a resource in one transaction. **Data and page :** designed the maintenance_records table, added the demo records to the seed file, and wrote the maintenance page. **Resource guard :** wrote the guard in resource_model.py so the Resource API cannot move a resource into or out of Maintenance. **Tests :** wrote the maintenance tests in test_maintenance.py, including the regression test for the lost row id. **Extra - Token invalidation :** wrote the password fingerprint in auth_middleware.py, so a token issued before a password change stops working.	**State transition rules :** a record moves Reported → Open / In Progress → Resolved, or Reported → Dismissed. The system checks the current state before saving, so a record cannot be confirmed or resolved twice. **Single source of truth :** the Maintenance status of a resource comes only from its maintenance records. The Resource API refuses to set or clear it, and the update is conditional, so an old edit form cannot overwrite it. **Deadlock avoidance :** the lock order is always the resource row first and then the record, the same order that booking and check-in use. **Relational database design :** foreign keys link each record to its resource, reporter, confirming administrator and closing administrator. The status is an ENUM, and an index on (resource_id, status) supports the "is this resource blocked?" query. **Regression testing :** after an UPDATE the saved id was reset to 0, so a saved report returned a 500. We read the id straight after the INSERT and added a test that reads the saved record back.	**6138927** maintenance_model.py **934967b** maintenance_controller.py **d303abc** Maintenance guard in resource_model.py **e80b3f3** demo records in the seed file **574207c 11d1f7d** maintenance and Report issue UI	5

Panyasiri Aimngern	**US-10 (In Use rule) :** wrote the "In Use" rule in booking_model.py. When someone is checked in, a new slot must start after their booking ends. When the status was set by hand and nobody is checked in, only a slot starting within the check-in window is refused. Maintenance blocks booking completely. **US-10 (approval re-checks) :** wrote the checks that run again when an administrator approves a booking: already started, resource archived, resource under Maintenance, resource In Use, and overlap with an approved booking. **Tests :** wrote test_booking_in_use.py and test_booking_reason.py. **Extra - Reject reason :** write the optional reason (up to 255 characters) that is saved with a rejection and shown to the student. **Extra - Platform :** wrote the JSON error handler and the JSON-object check in app.py, the type and length limits for resources and announcements, and the Docker health checks.	**Interval overlap :** two bookings overlap when each starts before the other ends, so back-to-back bookings stay allowed. At approval time only Approved bookings are compared. **Validation at the point of use :** a request that waited in the queue is checked again when it is approved, because the resource may have changed since the student submitted it. **Compare-and-set :** the decision is saved only while the booking is still pending, so two administrators who decide at the same moment cannot overwrite each other. **Lock ordering :** deciding a booking locks the resource row first and the booking second, the same order that creating a booking uses, so the two cannot deadlock. **Fail-safe API design :** every unexpected error on an /api route returns JSON with a general message and does not expose internal details.	**9417a8e** “In Use” rule, availability and conflict checks in booking_model.py **3aefe88** rejection reason in booking_controller .py **98f067d** approval and rejection-reason UI **39f2839** seed data **079354d** bounded integer env helper in config.py **fb33620** sprint 4 - add booking in use test **81f3250** sprint 4 - add booking reason test **01ca543** sprint 4 - API Error handlering	5